

Technische Anforderungen und ADRs im Code

WirSchiffenDas – Engine Quality Analysis

Theodoros Bampis & Yunus Emre Sirin

Sommersemester 2026

Service-basierte Komponenten-Architekturen (SEKA)

Teil 1 – Einordnung

- ▶ Ziel der Präsentation
- ▶ Traceability-Ansatz

Teil 2 – Architektur

- ▶ Service-Landschaft
- ▶ Choreographie

Teil 3 – Technische Anforderungen

- ▶ TA-01 bis TA-05 im Code

Teil 4 – Architekturentscheidungen

- ▶ ADR-01 bis ADR-06 im Code

Teil 5 – Demo & Fazit

Leitfrage

Welche Anforderung ist **wo** im Code nachweisbar – und **warum** wurde sie so umgesetzt?

Problem

- ▶ Architekturdokumente beschreiben Anforderungen.
- ▶ Ohne Code-Bezug bleibt offen, ob sie umgesetzt sind.

Lösung: Traceability

- ▶ Jede Anforderung → konkrete Datei + Symbol.
- ▶ Jedes Snippet → kurze Erläuterung.
- ▶ Nachvollziehbar in der mündlichen Prüfung.

Kennungen

Technische Anforderung
Entwurfsentscheidung
Funktionale Anforderung
Qualitätsanforderung

Quelle

`docs/code-traceability.md`

Traceability-Matrix (Auszug)

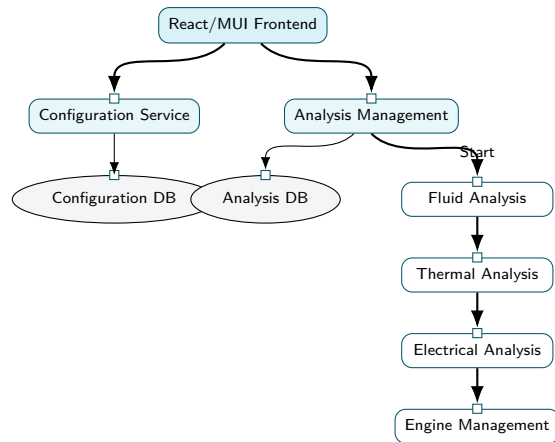
Kennung	Anforderung	Zentraler Code-Nachweis
TA-01	Spring Boot	@SpringBootApplication in 6 Service-Klassen
TA-02	Docker	6 × Dockerfile (Multi-Stage)
TA-03	Docker Compose	docker-compose.yml (6 Services, Volumes)
TA-04	Resilience4j	@CircuitBreaker + fallback + application.yml
TA-05	Kafka (COULD)	bewusst nicht umgesetzt (ADR-02)
ADR-01	Fachlicher Schnitt	Modulstruktur api/application/domain/infrastructure
ADR-03	Choreographie	AnalysisWorker → NextServiceClient
ADR-06	Keine Shared Persistence	ConfigurationSnapshot + stateless Services

Vollständige Matrix: docs/code-traceability.md

- ▶ **6 Microservices**, je ein Spring-Boot-Prozess.
- ▶ **2 zustandsbehaftet**: Configuration, Analysis Management.
- ▶ **4 stateless**: die Analyse-Services.
- ▶ Kommunikation ausschließSSlich über REST.

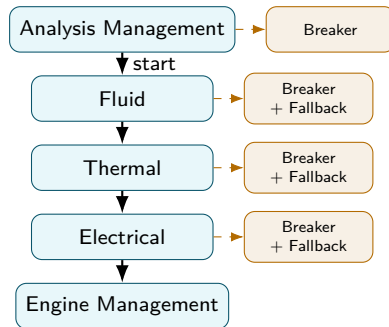
Fachlicher Schnitt

Jeder Service kapselt eine Business Capability – kein technischer Layer-Schnitt (ADR-01).



Ablauf

1. Analysis Management startet **nur** den Anchor FLUID.
2. Jeder erfolgreiche Service ruft selbst den nächsten auf.
3. Ergebnisse werden im AnalysisCommand mitgereicht.
4. Engine Management berücksichtigt Vorergebnisse.



Kernaussage

Kein zentraler Orchestrator – die Ablaufsteuerung liegt verteilt in den Services (ADR-03).

Circuit Breaker + Fallback

Fluid, Thermal, Electrical Breaker **mit Fallback** (markiert Folgeschritt als FAILED).
Analysis Management ohne Fallback (ADR-04).

```
1 @EnableAsync
2 @SpringBootApplication
3 public class FluidAnalysisServiceApplication {
4     public static void main(String[] args) {
5         SpringApplication.run(
6             FluidAnalysisServiceApplication.class, args);
7     }
8 }
9
```

services/fluid-analysis-service/.../FluidAnalysisServiceApplication.java

Erläuterung

- ▶ @SpringBootApplication aktiviert Auto-Configuration, Component Scan und die eingebettete Servlet-Engine.
- ▶ @EnableAsync ist die Grundlage für die asynchrone Ausführung (QR-08).
- ▶ Identisches Muster in allen sechs Services.

Abhängigkeit

pom.xml: spring-boot-starter-web

```
1 FROM maven:3.9.16-eclipse-temurin-21 AS build
2 WORKDIR /workspace
3 COPY pom.xml ./
4 COPY services ./services
5 RUN mvn -q -pl services/fluid-analysis-service \
6     -am clean package -DskipTests
7
8 FROM eclipse-temurin:21-jre
9 WORKDIR /app
10 COPY --from=build /workspace/services/fluid-analysis-service/target/
11     fluid-analysis-service-0.1.0-SNAPSHOT.jar app.jar
12 EXPOSE 8083
13 ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

services/fluid-analysis-service/Dockerfile

Erläuterung

- ▶ **Multi-Stage-Build:**
Build-Umgebung (Maven + JDK)
getrennt vom schlanken
Runtime-Image (nur JRE).
- ▶ Jeder Service erhält ein eigenes
Image und einen eigenen Port.
- ▶ Grundlage für Independent
Deployability (QR-03, ADR-05).

Nachweis

6 × Dockerfile – eines pro Service.

```
1 services:
2   configuration-service:
3     image: ysirin2s/seka-wirschaffendas:configuration-service-v${
4       VERSION}
5     ports:
6       - "8081:8081"
7     volumes:
8       - configuration-data:/app/data
9   analysis-management-service:
10    image: ysirin2s/seka-wirschaffendas:analysis-management-service-v${
11      VERSION}
12    ports:
13      - "8082:8082"
14    environment:
15      CONFIGURATION_SERVICE_URL: http://configuration-service:8081
16      FLUID_ANALYSIS_SERVICE_URL: http://fluid-analysis-service:8083
17    volumes:
18      - analysis-data:/app/data
```

`docker-compose.yml`

Erläuterung

- ▶ Compose definiert alle sechs Services, Ports, Umgebungsvariablen und getrennte Volumes.
- ▶ Services finden sich über **Service-Namen** im Netzwerk `wirschaffendas-network` – keine fest verdrahteten IPs.
- ▶ `depends_on` steuert die Startreihenfolge.

Start

`docker compose up -build`

```
1 @CircuitBreaker(name = "nextService", fallbackMethod = "fallback")
2 public void startNext(AnalysisCommand command, String currentResult) {
3     var results = new ArrayList<>(command.previousResults());
4     results.add(Map.of("algorithm", "FLUID", "result", currentResult));
5     AnalysisCommand nextCommand = new AnalysisCommand(
6         command.analysisId(), command.configuration(), results);
7     client.post().uri("/internal/analyses")
8         .body(nextCommand).retrieve().toBodilessEntity();
9 }
10 private void fallback(AnalysisCommand command,
11     String currentResult, Throwable throwable) {
12     managementClient.reportStatus(command.analysisId(), "THERMAL",
13         "FAILED", "thermal-analysis-service unavailable");
14 }
15
```

services/fluid-analysis-service/.../infrastructure/Next
ServiceClient.java

Erläuterung

- ▶ Der Breaker fängt Verbindungsfehler ab.
- ▶ Statt Fehlerkaskade läuft der **Fallback**: markiert den Schritt als FAILED.
- ▶ Der aufrufende Service bleibt erreichbar.
- ▶ Nach Wartezeit → Half-Open → Retry möglich.

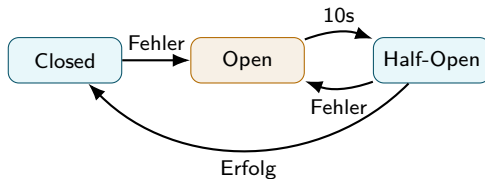
Wirkung

Isolation of Failures – der Fehler bleibt lokal (ADR-04).

```
1 resilience4j:
2   circuitbreaker:
3     instances:
4       nextService:
5         sliding-window-size: 2
6         minimum-number-of-calls: 1
7         permitted-number-of-calls-in-half-open-state: 1
8         failure-rate-threshold: 50
9         wait-duration-in-open-state: 10s
10        automatic-transition-from-open-to-half-open-enabled: true
11
```

services/fluid-analysis-service/src/main/resources/application.yml

Zustandsmodell



Health-Endpoint

/actuator/health zeigt den Breaker-Zustand.

Anforderung

- ▶ Priorität **COULD** – nicht Teil des Pflichtumfangs.
- ▶ Kafka könnte Statusnachrichten asynchron übertragen.

Entscheidung

- ▶ Kein Kafka im Code.
- ▶ Statusmeldungen laufen synchron per REST-Callback (FR-09).

Begründung (ADR-02)

Für den kleinen PoC ist REST einfach nachvollziehbar, testbar und mit der Aufgabenstellung kompatibel. Kafka bleibt bewusst auSSerhalb des Kernumfangs.

Trade-off

REST erzeugt temporale Kopplung. Für eine produktive Variante wäre asynchrones Messaging zu prüfen.

Entscheidung

- ▶ Services werden nach **Business Capabilities** geschnitten.
- ▶ Kein technischer Layer-Schnitt (kein “Controller-Service”).

Nachweis: Modulstruktur

```
1 services/  
2   configuration-service/      -> Engine-Konfiguration  
3   analysis-management-service/ -> AnalysisRun, Status,  
   Resultate  
4   fluid-analysis-service/     -> Oil-/Fuel-Analyse  
5   thermal-analysis-service/   -> thermische Analyse  
6   electrical-analysis-service/ -> elektrische Analyse  
7   engine-management-analysis-service/ -> EMS-Analyse  
8
```

Interne Schichtung (je Service)

```
1 fluid/  
2   api/           AnalysisController, AnalysisCommand  
3   application/   AnalysisWorker  
4   infrastructure/ NextServiceClient,  
5                   AnalysisManagementClient  
6   FluidAnalysisServiceApplication.java  
7
```

Wirkung

Vermeidung des Anti-Patterns **Wrong Cut** (QR-06).

```
1 @RestController
2 @RequestMapping("/internal/analyses")
3 public class AnalysisController {
4
5     private final AnalysisWorker worker;
6
7     @PostMapping
8     public ResponseEntity<Void> start(
9         @RequestBody AnalysisCommand command) {
10         worker.execute(command);
11         return ResponseEntity.accepted().build();
12     }
13 }
14
```

services/fluid-analysis-service/.../api/AnalysisController.java

Erläuterung

- ▶ Alle Services kommunizieren über HTTP/REST mit definierten DTOs.
- ▶ Kein Service greift auf interne Klassen eines anderen zu.
- ▶ Ausgehende Aufrufe über RestClient.

Nachweis

*Controller (eingehend) +
RestClient-Clients (ausgehend) in jedem Service.

Start nur des Anchor-Algorithmus

```
1 public AnalysisRun start(String configurationId) {
2     ConfigurationSnapshot configuration =
3         configurationClient.get(configurationId);
4     AnalysisRun run = AnalysisRun.start(
5         "A-" + UUID.randomUUID(), configurationId);
6     AlgorithmExecution fluid = run.execution(AlgorithmName.FLUID);
7     fluid.updateStatus(AnalysisStatus.RUNNING, null);
8
9     // Commit before external HTTP call (avoids callback race).
10    run = repository.saveAndFlush(run);
11
12    serviceStarter.start(AlgorithmName.FLUID,
13        new AnalysisCommand(run.getAnalysisId(),
14            configuration, List.of()));
15    return run;
16 }
17
```

.../application/AnalysisApplicationService.java

Weitergabe durch den Service

```
1 @Async
2 public void execute(AnalysisCommand command) {
3     managementClient.reportStatus(
4         command.analysisId(), ALGORITHM, "RUNNING",
5         null);
6     if (!pause(command.analysisId())) return;
7
8     boolean ok = valid(command.configuration()
9         .get("oilSystem"))
10        && valid(command.configuration()
11        .get("fuelSystem"));
12    String result = ok ? "OK" : "FAILED";
13    managementClient.reportResult(command.analysisId(),
14        ALGORITHM, ok ? "READY" : "FAILED", result,
15        null);
16
17    if (ok) {
18        nextServiceClient.startNext(command, result);
19    }
20 }
```

.../application/AnalysisWorker.java

```
1 @CircuitBreaker(name = "analysisServiceStarter")
2 public void start(AlgorithmName algorithm,
3                 AnalysisCommand command) {
4     String baseUrl = switch (algorithm) {
5         case FLUID -> fluidUrl;
6         case THERMAL -> thermalUrl;
7         case ELECTRICAL -> electricalUrl;
8         case ENGINE_MANAGEMENT -> engineManagementUrl;
9     };
10
11     builder.baseUrl(baseUrl).build()
12         .post().uri("/internal/analyses")
13         .body(command).retrieve().toBodilessEntity();
14 }
15
```

.../infrastructure/AnalysisServiceStarter.java

Erläuterung

- ▶ Umsetzung von **Isolation of Failures**.
- ▶ Der Fehler bleibt lokal im aufrufenden Service behandelbar.
- ▶ Zwei Breaker-Instanzen:
 - analysisServiceStarter (Management → Analyse)
 - nextService (Analyse → nächster Service)

Detail

Vollständiger Code + Fallback: siehe TA-04.

Entscheidung

- ▶ Jeder Service besitzt ein eigenes Dockerfile.
- ▶ Jeder Service läuft in einem eigenen Container.

Nachweis

- ▶ 6 × Dockerfile (Multi-Stage).
- ▶ `docker-compose.yml` mit 6 Services und Ports 8081–8086.

configuration-service :8081

analysis-management :8082

fluid-analysis :8083

thermal-analysis :8084

electrical-analysis :8085

engine-management :8086

Wirkung

Unterstützt Independent Deployability (QR-03).

Getrennte Entities

```
1 @Entity
2 @Table(name = "engine_configurations")
3 public class EngineConfiguration {
4     @Id
5     private String configurationId;
6     private String oilSystem;
7     private String fuelSystem;
8     // ...
9 }
10
```

configuration-service/.../domain/EngineConfiguration.java

```
1 @Entity
2 @Table(name = "analysis_runs")
3 public class AnalysisRun {
4     @Id
5     private String analysisId;
6     // ...
7 }
8
```

analysis-management-service/.../domain/AnalysisRun.java

Integration über DTO

```
1 public record ConfigurationSnapshot(
2     String configurationId,
3     String oilSystem,
4     String fuelSystem,
5     String coolingSystem,
6     String electricalSystem,
7     String engineManagementSystem) {
8 }
9
```

.../infrastructure/ConfigurationSnapshot.java

Nachweis stateless

Analyse-Services haben **keinen** spring.datasource-Block in application.yml.

Ablauf der Demo

1. System starten:

```
1 docker compose up --build
2
```

2. Thermal stoppen:

```
1 docker compose stop thermal-analysis-service
2
```

3. Analyse starten → THERMAL = FAILED.

4. Thermal starten:

```
1 docker compose start thermal-analysis-service
2
```

5. Retry ausführen → overallResult = OK.

Retry-Endpoint

```
1 POST /api/analyses/{analysisId}/algorithms/THERMAL/retry
2
```

Was die Demo zeigt

- ▶ **TA-04 / ADR-04:** Circuit Breaker fängt den Ausfall ab.
- ▶ **FR-11:** System bleibt erreichbar, Schritt wird FAILED.
- ▶ **FR-10:** Nur der fehlgeschlagene Schritt wird wiederholt.
- ▶ **ADR-03:** Choreographie läuft ab Thermal weiter.

```
1 public void recalculateOverallResult() {  
2     boolean failed = executions.stream()  
3         .anyMatch(e -> e.getStatus() == AnalysisStatus.FAILED  
4             || e.getResult() == AnalysisResult.FAILED);  
5  
6     if (failed) {  
7         overallResult = AnalysisResult.FAILED;  
8         return;  
9     }  
10  
11     boolean unfinished = executions.stream()  
12         .anyMatch(e -> e.getStatus() == AnalysisStatus.PENDING  
13             || e.getStatus() == AnalysisStatus.RUNNING);  
14  
15     overallResult = unfinished ? null : AnalysisResult.OK;  
16 }  
17
```

analysis-management-service/.../domain/AnalysisRun.java

Regel (architecture.md §8)

- ▶ Ein Fehler → FAILED.
- ▶ Schritte offen → null.
- ▶ Alle vier fertig → OK.

Test

AnalysisRunTest prüft alle drei Fälle (QR-07).

```
1 @PostMapping
2 public ResponseEntity<Void> start(
3     @RequestBody AnalysisCommand command) {
4     worker.execute(command);
5     return ResponseEntity.accepted().build();
6 }
7
```

.../api/AnalysisController.java

```
1 @Async
2 public void execute(AnalysisCommand command) {
3     // laeuft im Hintergrund-Thread
4 }
5
```

.../application/AnalysisWorker.java

Erläuterung

- ▶ Der Controller antwortet sofort mit 202 Accepted.
- ▶ `worker.execute` läuft dank `@Async` im Hintergrund.
- ▶ Der Client blockiert nicht bis zum Ende der Analyse.
- ▶ Status ist währenddessen weiter abfragbar (QR-02).

Zusammenspiel

`@EnableAsync` (TA-01) + `@Async` + 202 Accepted.

Technische Anforderungen

- ✓ TA-01 Spring Boot
- ✓ TA-02 Docker
- ✓ TA-03 Docker Compose
- ✓ TA-04 Resilience4j
- TA-05 Kafka (COULD, bewusst offen)

Architekturentscheidungen

- ✓ ADR-01 bis ADR-06 im Code belegt

Kernbotschaft

Jede zentrale Anforderung ist über eine konkrete Datei und ein Symbol im Code nachweisbar – nicht nur dokumentiert, sondern implementiert und testbar.

Referenzen

docs/code-traceability.md
docs/arc42.md
docs/requirements.md
scripts/e2e.sh

Vielen Dank!

Fragen und Diskussion

Theodoros Bampis & Yunus Emre Sirin
SEKA – Sommersemester 2026